BLDC方波过零点调试

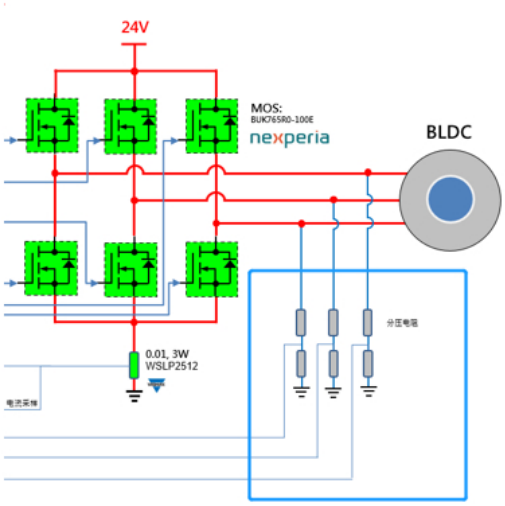
过零点概念理解

在每个相序，都会纯在电压从0到高这个过程，刚好这个过零点就存在，通用的办法就是ADC采样，采集母线电压的一半，这样就可以看作过零信号，但是要把成本降低下去，可以使用比较器方法，利用比较器的输出信号，就能很好的获取过零信号。以下是两种采集办法图示

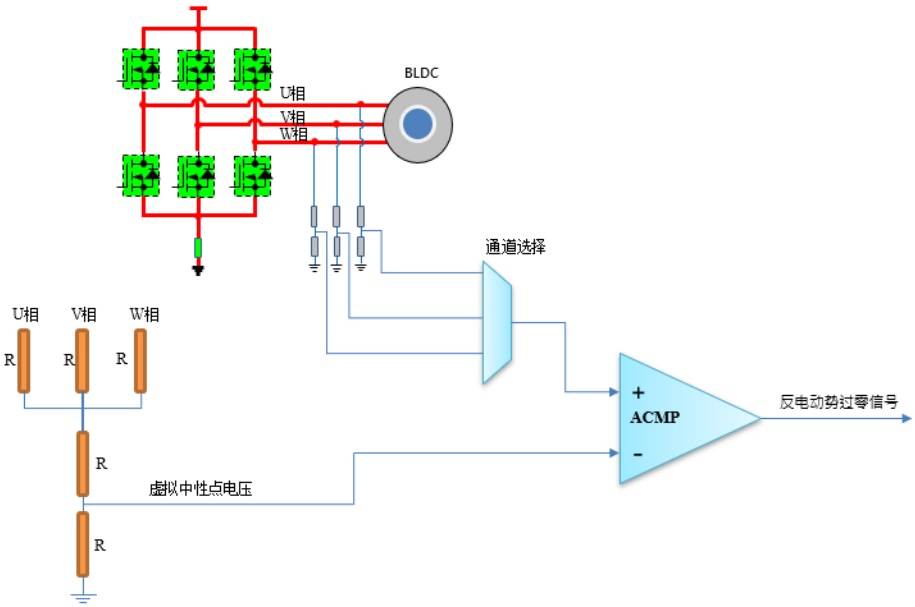
2. 过零检测硬件部分介绍

目前使用比较多的反电动势过零检测方法主要有：

1. 使用 ADC 同时采样母线电压和浮空相的相电压，采集到的相电压与母线电压的一半相减，当产生正过零或负过零时产生过零点，母线电压可以通过分压后采样，也可以在打开 PWM 时的相线上分压后采样，灵动微方案采用后一种，分压电阻如下图蓝框所示：



2. 使用比较器和虚拟中性点做过零检测，在星型连接的电机中，因其中点是不接线出来的，因此无法直接测量相电压， 采用如下方式创建虚拟中性点，中性点连接至比较器负端做比较电压基准点，比较器正端在电机转子处于不同扇区时切换到不同的输入通道上，或者使用三个比较器，灵动微方案使用的是三个比较器，程序中通过查询的方式来检测，无需切换比较器通道，整体信号连接如下框图所示：



其中，为什么过零点是母线电压的一半，有公式推导，电机电压平衡方程如图

当电机正常运行时，对于星型连接的三相电机电枢绕组的电压平衡方程式为：

$$Uun = R*Iu + L*\frac{dIu}{dt} + Eun \quad (1)$$

$$Uvn = R*Iv + L*\frac{dIv}{dt} + Evn \quad (2)$$

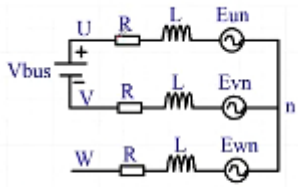
$$Uwn = R*Iw + L*\frac{dIw}{dt} + Ewn \quad (3)$$

以上3个式子中，Uun，Uvn，Uwn表示三相相电压，Iu，Iv，Iw表示相电流，R表示绕组内阻，L表示绕组电感量，Eun，Evn，Ewn表示三相反电动势。对于图三的等效电路来说，此时绕组的通电是U相接电源正，V相接电源负，W相不通电（悬空相），根据这个关系，我们可以得出以下条件：

$$Iu = -Iv \quad (4)$$

$$Iw = 0 \quad (5)$$

$$Iu + Iv + Iw = 0 \quad (6)$$

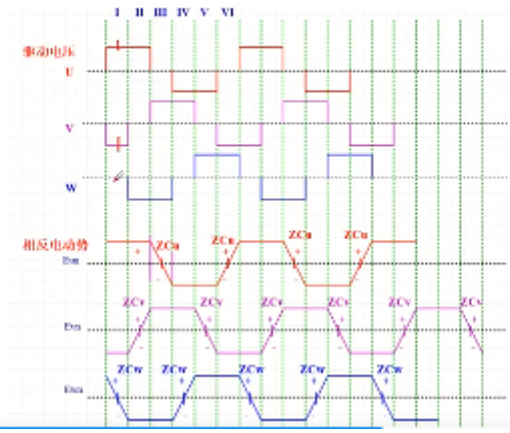


W相悬空时的等效电路

图三：等效电路图

反电动势和驱动电压的图

为了进一步分析，我们画出绕组的驱动电压跟反电动势波形，如下图四所示：



在任何过零点，可以得出，且与123式子联立化简

反电动势过零点检测方法

张飞实战电子
晚课带学 | 名师录课

从上图四，我们可以知道，对于具有梯形波反电动势的电机来说，在悬空相出现过零点时，有以下关系成立：

$$Eun + Evn + Ewn = 0 \quad (7)$$

所以，在悬空相（W相）反电动势过零点的位置处，结合电压平衡方程式及已知条件(4)(5)(6)(7)，可以推出：

由(7)式可以得出：

$$Uun + Uvn + Uwn = Eun + Evn + Ewn$$

将(8)式拆分，可以得到：

$$Uun + Uvn + Uwn = 0 \quad (8)$$

$$Uu - Un + Uv - Un + Uw - Un = 0 \quad (9)$$

那么Uu-Un就是从U到中性点的电压，再单独算Uw端 Iw为0

最后可以得出结论当W相出现反电动势过零点的时候，Uw=vbus/2

注意，此时是根据Uw的电压去推断过零点。

(9)式中，Uu，Uv，Uw表示电机U，V，W三相线对地的电压（端电压），Un表示三相电机星型连接点n对地的电压。对(9)式进行整理，可以得到：

$$Un = \frac{Uu + Uv + Uw}{3} \quad (10)$$

此时对悬空相(W相)求解端电压，则有：

$$Uwn = R*Iw + L*\frac{dIw}{dt} + Ewn$$

$$Uwn = Ewn$$

$$Uw - Un = Ewn \quad (11)$$

当出现悬空相(W相)出现反电动势过零点时，可以推导得出以下最终式(12)的结论：

$$Uw - Un = Ewn$$

$$Uw - \frac{Uu + Uv + Uw}{3} = 0$$

$$Uu = Vbus，Uv = 0$$

$$Uw - \frac{Uw}{3} - \frac{Vbus}{3} = 0$$

$$\frac{2*Uw}{3} - \frac{Vbus}{3} = 0$$

$$2*Uw = Vbus$$

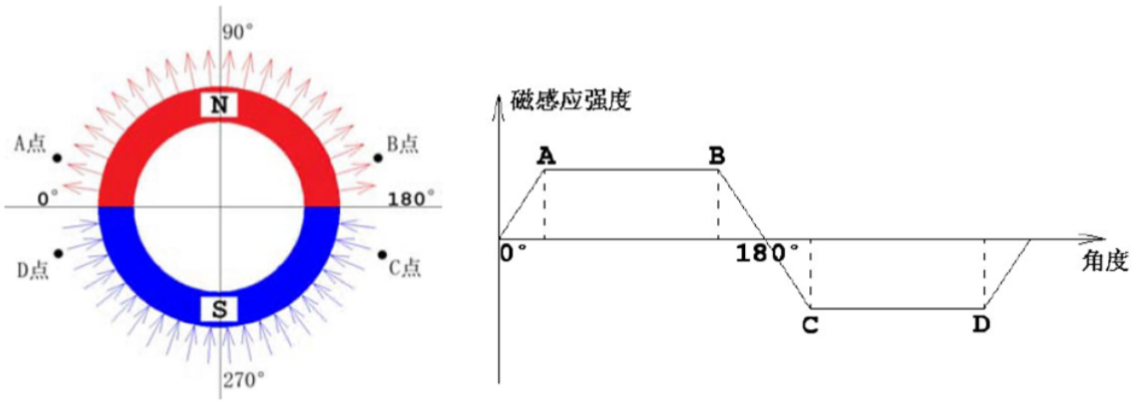
$$Uw = \frac{Vbus}{2} \quad (12)$$

按照同样的方式，对U，V两相中出现悬空相时，也能得出与式(12)相同的结论。

如果采用比较器的方法，首先考虑原理

磁感应强度是在AB之间为最大

下图展示了内转子磁极的磁感应强度B的分布情况。定义磁感应强度方向向外为正



在0°的时候，处于正反方向交界处，**磁感应强度为零**；

然后开始**线性增加**，在**A点**时达到最大

然后一直保持恒定值不变，直到**B点**开始下降，到**180°**的时候下降到零。

然后开始负向增长，在**C点**处达到负值最大，然后保持恒定负值不变

直到D点强度开始减弱，到0°时又回到零。

但是，如果是反电动势最大的点，其实是在磁感应强度变化最大的点，也就是斜率最大的点，用E=BLV这个公式可以得知

AA BB CC都会产生反电动势，在AB通电的期间，C相是悬空，但是有反电动势磁产生

2、反电动势过零的基本原理

在AB相通电时，第三相**C相会产生一个反电动势（可利用这个第三相的感应电动势来测算转子的位置）**；

在AB相通电期间，如果我们去测量线圈CC’ 上的电压，会发现**CC’ 上的电压**其间有一个**从正到负的变化过程**；

在AB相通电期间，不只是线圈CC’ 上产生感生电动势，其实**AA’ 和BB’ 也在切割磁力线，也都会产生感生电动势**，其电动势方向**与外加的12V电源相反**，所以叫**“反向感生电动势” （BEMF）**。其等效电路图见图1-25。

从图1-25可以看出，线圈绕组AA’ 和BB’ 上产生的**反电动势是很大的**，两个加起来几乎略小于12V。**$V_{AA'}+V_{BB'}\approx 12V$。**

因为线圈绕组本身的等效电阻很小（$R_{AA'}=R_{BB'}\approx 0.1\Omega$），**如果反电动势不大的话，端电压加载在线圈绕组等效电阻上，会产生巨大的电流，线圈非烧掉不可。**

例：假设在额定转速下**AA’ 和BB’ 各产生5.7V的反电动势**，那么它们串联起来就产生11.4V的反电动势，结合图1-25看，那么加载在等效电阻上的电压就为12-11.4=0.6V，最终通过绕组AB的电流就是0.6/(2X0.1)=3A，看来这个假设还是比较合理的。

同理，由于各绕组的结构是相同的，切割磁力线的速度也是相同的，所以线圈**CC’ 也应该会产生一个大小约为5.7V的感生电动势**；不同的是：**在AB相通电期间，CC’ 的感生电动势会整个换一个方向**，也即所谓的**“过零点”**。

也就是等效出来一个电源

相通电期间，CC’的感生电动势会整个换一个方向，也即所谓的“**过零点**”。

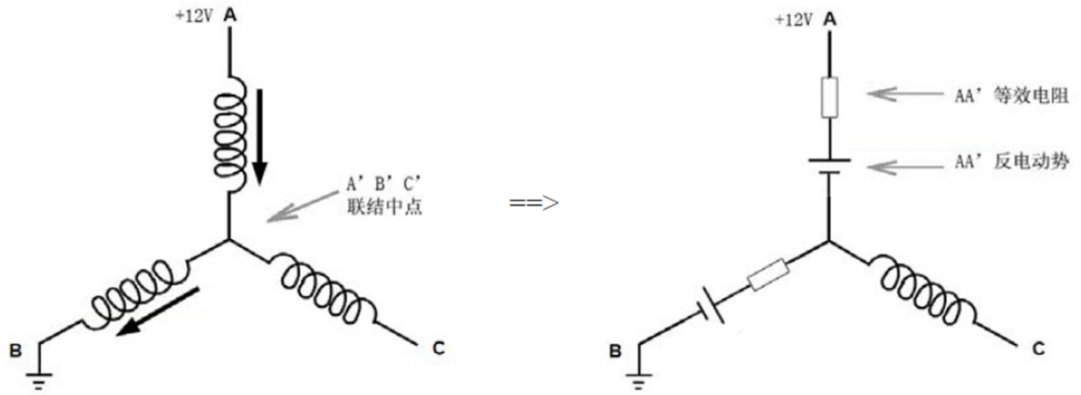


图 1-25 AB 相通电期间线圈 AA’和 BB’的等效电路（修改自 Microchip AN885 文档）

在图1-24的t_0时刻（图1-26(a)的位置），为**AB相通电刚开始时**的情况，CC’产生的感生电动势的等效电路图如图1-26(a)所示；**（CC’上无电流）**

在图1-24的t_1时刻（图1-26(b)的位置），为**AB相通电快结束时**的情况，CC’产生的感生电动势的等效电路图如图1-26(b)所示。**（CC’感应电动势方向改变）**

AB通电快结束的话，就是指t1时刻，感生电动势突然取反，因为他要维持这个方向电流

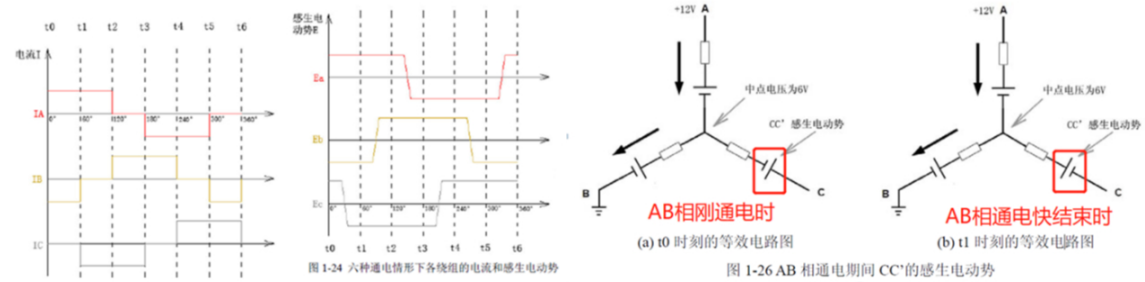


图 1-24 六种通电情形下各绕组的电流和感生电动势

(a) t0 时刻的等效电路图

(b) t1 时刻的等效电路图

图 1-26 AB 相通电期间 CC’的感生电动势

2-1 换相时机

由于中点电势值始终为6V，**CC’的线圈产生的感生电动势只能在以中点6V电势为基准点的基础上叠加；**

仍旧假设在额定转速下CC’上会产生5.7V的感生电动势

在t_0时刻，如果我们去测量C点的电压，其值应为6+5.7=11.7V；

在t1时刻，C点的电压值应为6-5.7=0.3V

如果电调的MCU足够快的话，可以采用连续AD采样的方式来测量C点电压，不过貌似有点浪费，因为大部分采到的AD值都是没用的，我们**只关心它什么时候低于6V。**

2-3 CMP采集过零信号

这时候模拟比较器的作用就来了。一旦C相输出电压低于6V，比较器马上可以感知并在输出端给出一个**下降沿**。同理，当电机处于AC相通电时，监测的是B相输出电压；当电机处于BC相通电时，监测的是A相输出电压。

继续往前，当电机开始进入BA相通电时，**C相输出电压一开始会处于一个较低的状态（0.3V），过零事件发生时，C相输出电压会超过6V**，也就是说，这时比较器会感知并输出一个**上升沿**。CA，CB相通电情况也类似。

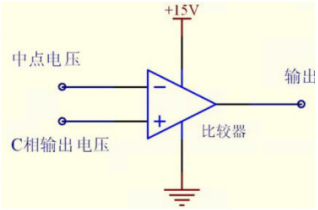


图 1-27 比较器的电路图

这是15V的比较器，单片机自带的比较器一般只支持最高5V的比较。上面这个电路图只是为了方便说明问题，**在真正的实用中，会对C相输出电压和6V中点电压再加个分压电路，而且中点电压也不总是等于6V**

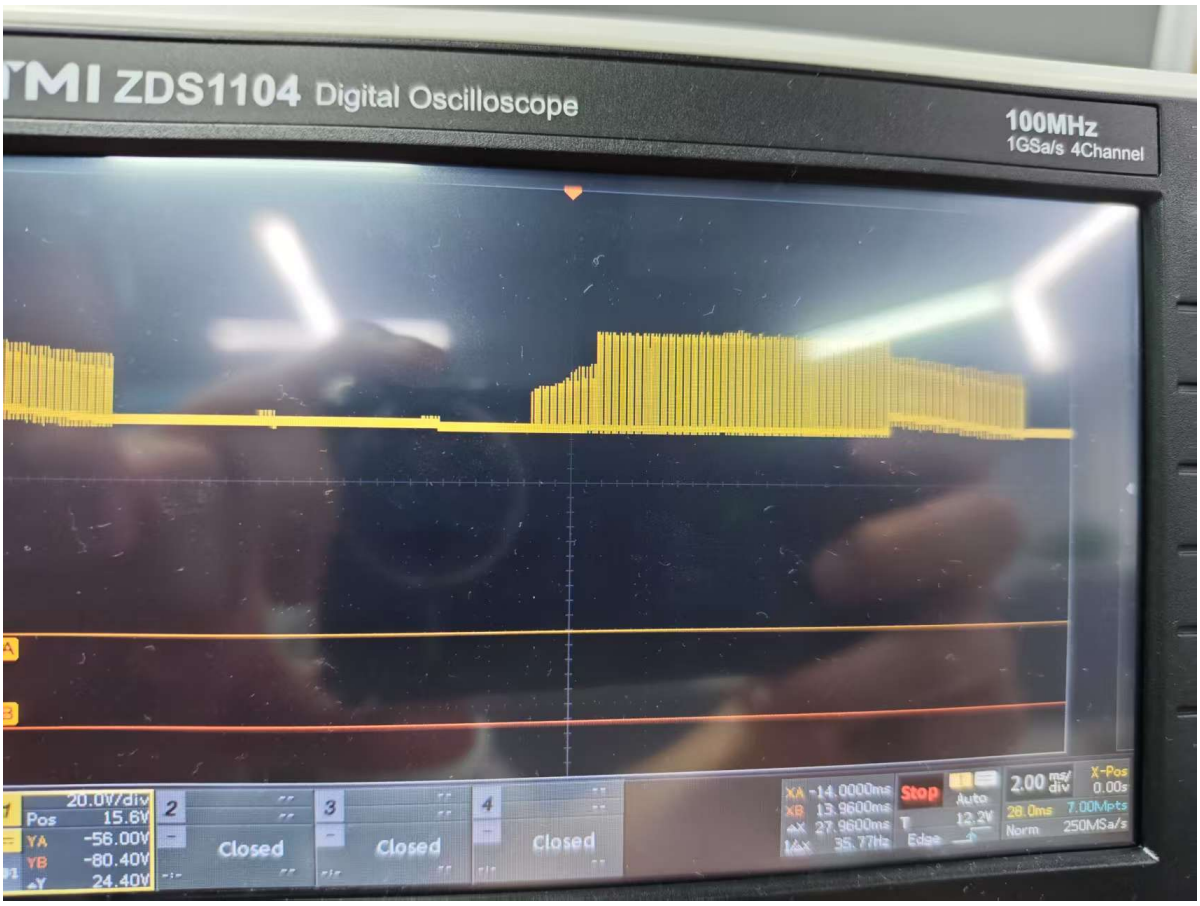
在实际中，我们需要构造一个虚拟中性点，且这个中性点的点位不一定是1/2VDC，因为会采用分压电阻把这个电阻值采上，并且，这个虚拟中性点指的是反电动势的中性点，并不是实际电机的中性点，用比较器，就是比较这个反电动势是否大于或小于这个反电动势中点

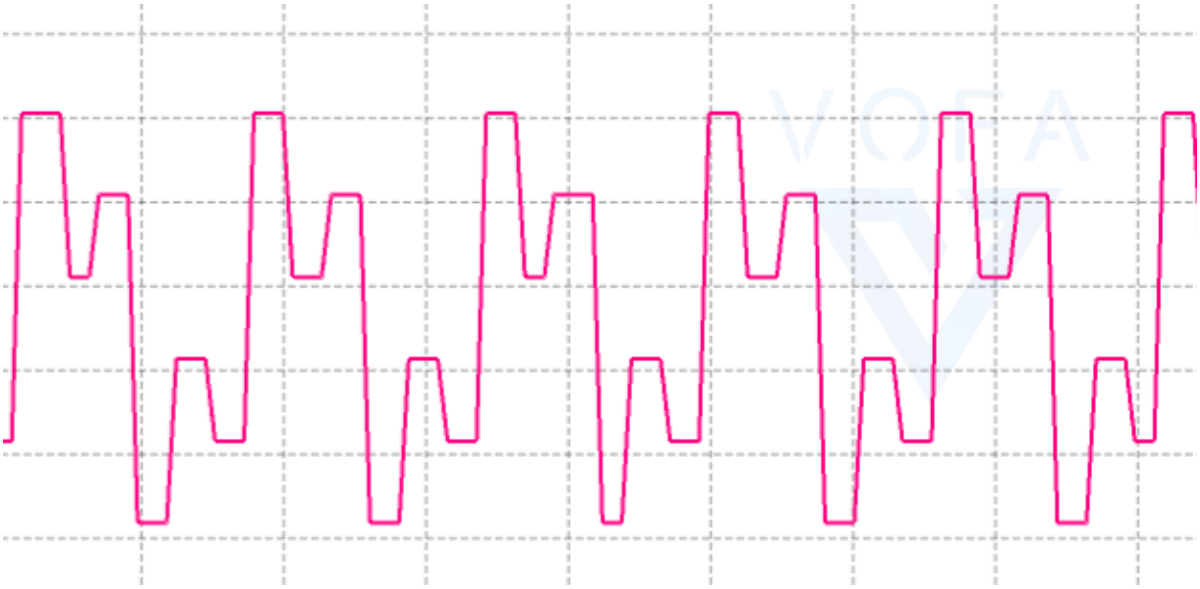
图中可以看到，这个反电动势采样点就是超出24V，并且当前中点为12V，但是经过分压电阻去采样，不是实际进入12v，在这个判断是否过零点

在调试的过程中，发现降频率降低，降到合适的频率，虽然不会出现很完美对称的UVW相电压，但是发现之前的尖峰信号都消失，并且此时，每次进入中断去读取比较器的值，再经过简单的滤波，可以发现稳点的过零信号

此时串口1ms发送一次数据

根据我的理解，比较器不需要开启滤波，比较器全速跑，因为比较器本身不用到他的中断，在我每次进入中断的时候，读取比较器的状态，此时判断01 然后累加这个状态，根据这个状态来区别是否达到过零，假设累加16个1状态，那么就判断出来过零





g_motor1_hallless.edge_queue[0] = g_motor1_hallless.edge_queue[0] <<1;
g_motor1_hallless.edge_queue[1] = g_motor1_hallless.edge_queue[1] <<1;
g_motor1_hallless.edge_queue[2] = g_motor1_hallless.edge_queue[2] <<1;

```
    g_motor1_hallless.edge_queue[0] |= COMP_GetOutStatus(COMP2);
    g_motor1_hallless.edge_queue[1] |= COMP_GetOutStatus(COMP4);
    g_motor1_hallless.edge_queue[2] |= COMP_GetOutStatus(COMP3);

    //U
    g_motor1_hallless.edge_filter_result
=g_motor1_hallless.edge_queue[0]&FILTER_LONG;
    if(g_motor1_hallless.edge_filter_result == FILTER_LONG)
    {
```

```
        g_motor1_hallless.queue_filter[0]=1;
    }
    else if (g_motor1_hallless.edge_filter_result == 0x00)
    {
        g_motor1_hallless.queue_filter[0]=0;
    }
    else
    {
        g_motor1_hallless.edge_filter_count++;
    }

    //V
    g_motor1_hallless.edge_filter_result
=g_motor1_hallless.edge_queue[1]&FILTER_LONG;
    if(g_motor1_hallless.edge_filter_result == FILTER_LONG)
    {
        g_motor1_hallless.queue_filter[1]=1;
    }
    else if (g_motor1_hallless.edge_filter_result == 0x00)
    {
        g_motor1_hallless.queue_filter[1]=0;
    }
    else
    {
        g_motor1_hallless.edge_filter_count++;
    }

    //W
    g_motor1_hallless.edge_filter_result
=g_motor1_hallless.edge_queue[2]&FILTER_LONG;
    if(g_motor1_hallless.edge_filter_result == FILTER_LONG)
    {
        g_motor1_hallless.queue_filter[2]=1;
    }
    else if (g_motor1_hallless.edge_filter_result == 0x00)
    {
        g_motor1_hallless.queue_filter[2]=0;
    }
    else
    {
        g_motor1_hallless.edge_filter_count++;
    }

    g_motor1_hallless.bemf_current_state=(g_motor1_hallless.queue_filter[0])|
(g_motor1_hallless.queue_filter[1]<<1)|(g_motor1_hallless.queue_filter[2]<<2);
```